How I would break this

Scope: Linux/ext4, honest fsync, abrupt process SIGKILL. Review against the current code; measured evidence is in MANUAL_TEST_REPORT.md and docs/GAP_FIX_REPORT.md. The printable companion reproduces this text in two pages.

1. Crash points

Locations are exact functions; engine means src/engine.py, serve means src/serve.py. Complete frames are the commit unit; G means the optional 00 EOF guard. Named hooks exercise boundaries, not every kernel instruction.

Point and function	Disk at interruption	Next open
init_before_rename; engine._create	Synced .init; destination not installed	Remove stale temp; initialize again
before_write; engine._write	Prior complete frames + G	Replay prior state
torn_write; engine._write	Partial new frame replaces G	Drop incomplete tail; restore G; sync
after_write; engine._write	Full frame + G; no successful fsync yet	Replay complete frame; no power-cut claim
after_fsync; engine._write	Frame + G synced; index not published	Replay durable operation
after_return; serve.execute	Engine returned; ACK not printed	Operation survives without observed ACK
recovery_mid_scan; engine._scan	Original bytes; partial RAM index	Rebuild index from the beginning
recovery_before_truncate; engine._recover	Valid prefix + torn tail	Repeat truncation/guard repair
recovery_after_truncate; engine._recover	Repaired prefix + G already synced	Replay repaired file
compact_mid_copy; engine._copy_live	Old file; incomplete or complete temp	Use old file; remove temp
compact_before_rename; engine._compact_locked	Old file; complete synced temp	Use old file; remove temp
compact_after_rename; engine._compact_locked	New file installed; directory not synced	Use complete new file under process-kill model
compact_after_swap; engine._compact_locked	Directory synced; new descriptor/index	Use new file; later writes use new descriptor
close_mid; engine.close	Acknowledged frames already synced	Replay them; close adds no commit data

2. Ordering argument

Append: validate/cap check, os.pwrite whole frame, os.pwrite G, os.fsync(file), publish index, return, then ACK. A complete frame is replayable even if index publication or the reply is interrupted. Recovery calls os.ftruncate for a torn tail, restores G when space allows, then os.fsync before readiness.

Compaction: write live frames + G to .compact, fsync and close it, os.replace over the database, fsync the parent directory, then reopen/swap descriptor and index. Creation uses the same temp-sync/replace/directory-sync order. Atomic replacement is assumed on tested ext4; process death leaves the old or complete new authority. Rename itself is not interrupted inside the kernel by a hook.

Append/sync errors poison the handle and attempt truncate/guard restoration plus fsync rollback. If rollback fails, UNCERTAIN allows the operation to survive or disappear; no ACK is sent. A failure after compaction replacement poisons a handle whose old descriptor is obsolete. File sync covers append contents/size; directory sync covers replacement of a name.

3. Known weaknesses

1. Recovery scans the entire log; long logs or laptop load can exceed two seconds. Compaction pauses readers/writers under one mutex. The RAM index grows with live keys.

2. Encoding is chunked, but input batches and queues retain values. The combined 10,000 distinct 1 MiB-value case (about 10 GiB input) is not qualified; no global byte admission budget exists.

3. Legacy logs already at a newly selected cap may have no room for G. They remain readable, but need space and a current write/compaction before one-byte-tail protection applies. Older engines may strip G.

4. The concurrent oracle stops searching after 250,000 states. Some errors, including duplicate dump keys, become TOOL_ERROR and allow later rounds; final exit is nonzero. Ordinary torture does not enforce the two-second gate; separate benchmarks do. Torn-prefix random draws are not seeded by the runner.

5. CRC32 is not authentication; arbitrary corruption and physical power failure are unqualified. check/inspect do not lock a mutating file, so their output is not a concurrent snapshot. The stable .lock file is intentional.

4. What we tried that failed

- Literal one-byte truncation previously lost the last acknowledged record. The synced expendable guard fixed F4; put/delete/batch/repeated restoration regressions verify it. Evidence: docs/GAP_FIX_REPORT.md.

- The old shared-batch checker accepted mixed results that no whole-operation ordering allowed. The exact ordering search and tests/test_shared_batches.py now reject those states.

- Scanning inside arbitrary value bytes could mistake embedded frame-like data for later records. Boundary-based scanning and tests/test_scan_boundaries.py fixed that false diagnosis; it is not a current known defect.

- Zero-progress pwrite could hang. The progress guard now raises an error. Isolated detector trials also reject deliberately early ACKs, partial batches and skipped tail repair (tests/test_crash_campaign.py).

- A loaded WSL disk exceeded the old five-second kill wait; a gate also ran out of host space. The wait is now 60 seconds with slow-exit reporting, and qualification checks host-volume reserves. Surviving tool errors never count as PASS.

5. If we had 24 more hours

1. Make judge runs fail fast on semantic output/tool failures, record/seed torn-prefix cuts, and expose persistent-file and strict recovery checks through one clear command.

2. Add byte-based admission limits for retained inputs/queues; qualify distinct maximum-sized values under an explicit RAM budget.

3. Add a crash-safe checkpoint or bounded replay strategy; test long logs and varied cache/load conditions against the strict two-second gate.